FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES

Ingeniería de Sistemas

BASES DE DATOS II

SQL Procedural

Triggers • Procedimientos • Funciones PL/pgSQL



Bases de Datos II

Limitaciones & SQL Procedural

FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES

Limitaciones de SGBD

Los SGBD no siempre brindan recursos declarativos para resolver problemas complejos:

- Implementar restricciones de tabla complejas.
- Establecer restricciones generales (assertions).
- Implementar acciones específicas de reparación o actualización.
- Otros requerimientos particulares...

Recurso: SQL Procedural

Lenguaje procedural (PL/SQL) para escribir *Persistent Stored Modules*:

Triggers Implícito

Código que se dispara automáticamente ante un evento para ejecutar acciones de control y sentencias SQL.

Stored Procedures Explícito

Código estructurado que se almacena en la base de datos y requiere una invocación directa.

Functions

Predefinidas o de usuario. Pueden invocarse de manera explícita o desde un trigger o procedimiento.

Bases de Datos II

Disparadores (TRIGGERS)

FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES

Especifican acciones automáticas que realiza el SGBD ante determinados eventos y condiciones. Corresponden a la funcionalidad de las **Bases de Datos Activas** y siguen el modelo de **reglas ECA** (Evento - Condición - Acción).

E • Eventos

Activan la regla de manera automática.

Corresponden generalmente a operaciones de actualización o modificación en la base de datos (como **INSERT**, **UPDATE**, o **DELETE**).

C • Condición

Opcional

Determina si la acción asociada debe ejecutarse.

Si se especifica, primero se verifica y evalúa. La acción solo se ejecuta si el resultado es **Verdadero**.

A • Acción

Se ejecuta automáticamente.

Suele ser una secuencia de sentencias SQL y puede incluir estructuras de control complejas procedimentales como **IF**, **WHILE** o **FOR**.

Cada DBMS fue proponiendo su propia solución procedural hasta que el estándar **SQL-1999** incorporó formalmente estas características, adaptando las soluciones previas al modelo común.

Sintaxis de Triggers (PostgreSQL)

FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES

postgresql_trigger.sql

```
CREATE [OR REPLACE] TRIGGER nombre
{ BEFORE | AFTER | INSTEAD OF }
{ evento [ OR ... ] }
evento = INSERT, UPDATE [OF col], DELETE,
TRUNCATE
ON tabla
[ FOR { EACH ROW | STATEMENT } ]
[ WHEN ( condición ) ]
EXECUTE FUNCTION func_especifica();
```

Componentes Clave

Tiempo de activación
Define si la acción se ejecuta antes, después o en lugar del evento.

Evento disparador
Operaciones DML: INSERT, UPDATE, DELETE o TRUNCATE.

Granularidad
EACH ROW (por cada fila) o STATEMENT (por sentencia SQL completa).

Condición (Opcional)
Expresión lógica evaluada antes de disparar la acción principal.

Acción
Invocación a la función PL/pgSQL que contiene la lógica procedural.

Nota de compatibilidad: PostgreSQL admite indistintamente **"EXECUTE PROCEDURE"** o **"EXECUTE FUNCTION"** (desde la versión 11).

Componentes de un Trigger

FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES

E • Evento

Operación sobre la tabla o vista a la que está asociado.

INSERT
Inserción de nuevos datos.

UPDATE
Modificación (permite especificar columnas).

DELETE
Eliminación de registros.

T • Activación

Momento de ejecución respecto a la sentencia disparadora.

BEFORE
Antes de ejecutar la sentencia.

AFTER
Después de ejecutar la sentencia.

INSTEAD OF
En lugar de la sentencia (sólo vistas).

G • Granularidad

Determina la frecuencia con la que se dispara el trigger.

FOR EACH ROW
Se ejecuta una vez por cada fila que resulta afectada por la operación.

FOR EACH STATEMENT
Se ejecuta una sola vez por sentencia SQL completa (opción por defecto).

Los disparadores permiten automatizar la lógica de negocio directamente en el motor de la base de datos.

Bases de Datos II

Sintaxis de Funciones (PostgreSQL)

FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES

postgresql_function.sql

```
CREATE [OR REPLACE] FUNCTION
    nombre_func()
RETURNS trigger AS $$
[DECLARE declaraciones de variables]
BEGIN
  Código PlpgSQL
END;
$$ LANGUAGE 'plpgsql';
```

Estructura Modular

PostgreSQL separa la declaración del **trigger** de la **función** que implementa la acción.

(En otros motores de BD, la acción se define directamente dentro del disparador)

Transaccionalidad

¡Importante! Si cualquier sentencia dentro del código de la función falla, la acción completa se deshace.

Esto incluye la reversión completa de la sentencia que disparó originalmente el trigger.

La arquitectura desacoplada de PostgreSQL ofrece mayor flexibilidad y reutilización de código procedural.

Variables en Triggers: NEW y OLD

FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES

NEW

Tipo de dato: `RECORD`

Variable que almacena la **nueva fila** para las operaciones de modificación.

- ✓ Válido en operaciones: **INSERT** y **UPDATE**
- ✓ Solo disponible en triggers a nivel de fila (*ROW*)

En triggers a nivel STATEMENT su valor es NULL

OLD

Tipo de dato: `RECORD`

Variable que almacena la **fila antigua** antes de las operaciones de modificación.

- ✓ Válido en operaciones: **UPDATE** y **DELETE**
- ✓ Solo disponible en triggers a nivel de fila (*ROW*)

En triggers a nivel STATEMENT su valor es NULL

¡Importante!

La función que implementa el trigger debe retornar `new`, `old` o `null`, según corresponda para la operación ejecutada.

Variables Especiales en Triggers

FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES

En PostgreSQL, disponemos de variables especiales de tipo `text` dentro de la función del trigger:

TG_NAME
Contiene el nombre del trigger actualmente disparado.

TG_WHEN
Especifica el momento de disparo: cadena con **BEFORE** o **AFTER** según la definición.

TG_LEVEL
Indica la granularidad del disparador: **ROW** o **STATEMENT**.

TG_OP
Operación que disparó el trigger: **INSERT**, **UPDATE** o **DELETE**.

TG_TABLE_NAME
Contiene el nombre de la tabla sobre la cual se activó el disparador.

Triggers: Casos de Utilidad

FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES

01

Forzar reglas de integridad o de negocio complejas

Útil cuando no es posible incluirlas de forma declarativa o se requieren acciones específicas de reparación.

02

Restricciones de transición de estados

Aplicable cuando se requiere conocer y comparar el valor anterior (OLD) y nuevo (NEW) de los datos.

03

Generación de logs e historial de cambios

Permite dar soporte de auditoría registrando de forma automática las acciones realizadas sobre la base de datos.

04

Mantenimiento de datos derivados

Facilita la propagación automática de actualizaciones y la generación dinámica de valores calculados.

05

Sincronización de vistas materializadas

Permite mantener vistas actualizadas de forma manual en escenarios donde el DBMS no provee soporte nativo.

Triggers: Reglas de Integridad

FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES

Forzar Reglas de Integridad (RI)

Útil para forzar las reglas de integridad que no pueden ser especificadas de forma declarativa.

¿Cómo transformar las RI declarativas en procedurales?

01. GRANULARIDAD
¿Qué tipo de granularidad debe tener un trigger que controla una RI?

02. TIEMPO DE ACTIVACIÓN
¿Qué tiempo de activación (BEFORE/AFTER) debe tener el trigger?

03. ORDEN DE DISPARO
Considerar de forma estricta el orden de disparo de los triggers...

Tabla / Evento	INSERT	UPDATE	DELETE
nombre_tabla	no / si	no / si (¿qué atributos?)	no / si

Bases de Datos II

Ejecución de los Triggers

FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES



El motor de base de datos asegura la consistencia transaccional garantizando el aislamiento de estados antes y después de la ejecución.

Modelo de Ejecución (SQL Estándar)

Ante una sentencia SQL de actualización, el SGBD determina las filas afectadas y aplica el siguiente orden:

Reglas y Consistencia

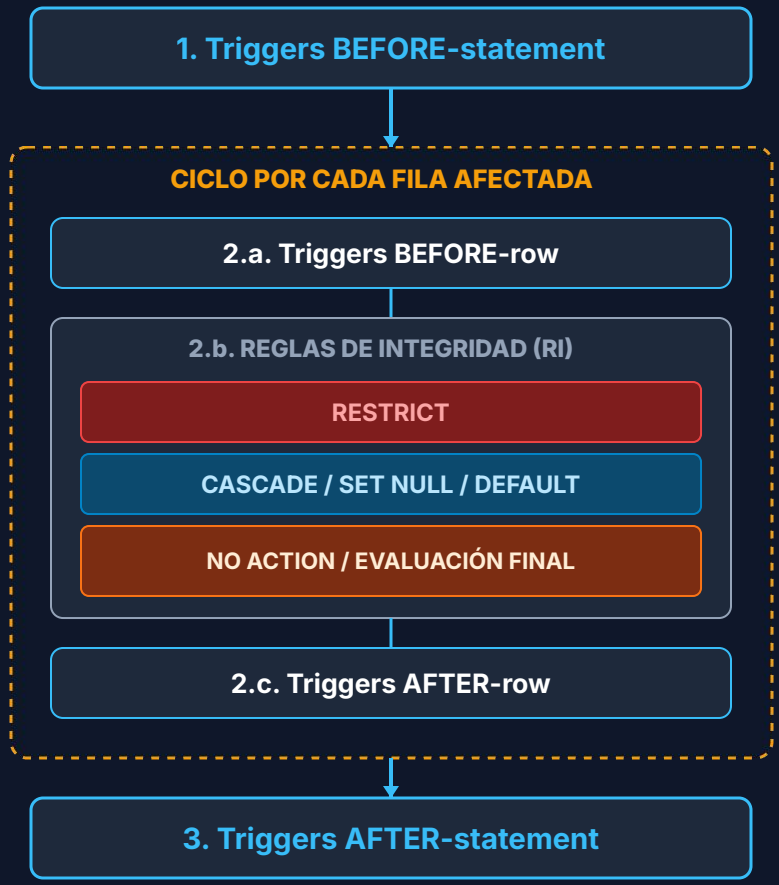
Fase 2: RI Declarativas (Orden estricto)

- **RESTRICT:** Se evalúa inmediatamente al intentar modificar la fila.
- **CASCADE / SET NULL:** Propagación de cambios en cascada.
- **NO ACTION:** Validación al finalizar la instrucción (diferible).

Consistencia Transaccional

Antes y después de la ejecución completa de la sentencia, el motor garantiza que la base de datos se mantiene en un **estado consistente**.

Nota: Puede que no todos los SGBD comerciales sigan estrictamente este esquema estándar.



Bases de Datos II

Implementación de RI Declarativa

FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES

Ejercicio: Los artículos pueden tener como máximo **15 palabras claves**.

Solución con Restricción de Tabla

```
ALTER TABLE CONTIENE
ADD CONSTRAINT CK_MAXIMO_PL_CLAVE
CHECK ( NOT EXISTS (
     SELECT 1
     FROM CONTIENE c
     GROUP BY id_articulo
     HAVING COUNT(*) > 15
));
```

Modelo de Relaciones

ARTICULO
id_articulo [PK] int
titulo, autor, fecha_pub...

⇅

CONTIENE
id_articulo [PK, FK] int
codigo_palabra [PK, FK] smallint

Tabla / Evento	INSERT	UPDATE	DELETE
CONTIENE	**SÍ**	**SÍ** (id_articulo)	**NO**

Bases de Datos II

Implementación de RI con Trigger

1. Función del Trigger (PL/pgSQL)

```
CREATE OR REPLACE FUNCTION FN_MAXIMO_PL_CLAVE()
RETURNS TRIGGER AS $$
BEGIN
  IF (EXISTS (SELECT 1 FROM CONTIENE
          WHERE id_articulo = new.id_articulo
          GROUP BY id_articulo
          HAVING COUNT(*) > 15)) THEN
    RAISE EXCEPTION 'El artículo supera la
    cantidad de palabras';
  END IF;

  RETURN NEW;
END $$ LANGUAGE 'plpgsql';
```

Disparadores BEFORE / AFTER

// retornar NEW u OLD en un trigger de tipo BEFORE es mandatorio para permitir o modificar la operación.

2. Definición del Trigger

```
CREATE TRIGGER
TR_MAXIMO_PL_CLAVES
AFTER INSERT OR UPDATE OF
  id_articulo
ON CONTIENE
FOR EACH ROW
EXECUTE FUNCTION
FN_MAXIMO_PL_CLAVES();
```

Lógica de Control

Este trigger de tipo **row-level** intercepta modificaciones en la tabla **CONTIENE**.

Valida dinámicamente si el artículo modificado excede el límite establecido de **15 palabras clave**.

Si se supera la restricción, aborta la transacción lanzando una excepción controlada.

Bases de Datos II

Implementación de RI con Trigger

1. Función del Trigger (PL/pgSQL) — Variante

```
CREATE OR REPLACE FUNCTION FN_MAXIMO_PL_CLAVES()
RETURNS TRIGGER AS $$
BEGIN
  IF ((SELECT count(*) FROM CONTIENE c
       WHERE id_articulo = new.id_articulo) > 15)
       THEN
    RAISE EXCEPTION 'El artículo supera la cantidad de
    palabras';
  END IF;

  RETURN NEW;
END $$ LANGUAGE 'plpgsql';
```

Comparación de Enfoques

// Esta variante calcula el conteo directo con count(*) en lugar de verificar la existencia con EXISTS. Ambos enfoques resuelven la restricción de integridad de manera eficaz.

2. Definición del Trigger

```
CREATE OR REPLACE TRIGGER
TR_MAXIMO_PL_CLAVES
AFTER INSERT OR UPDATE OF
id_articulo
ON CONTIENE
FOR EACH ROW
EXECUTE PROCEDURE
FN_MAXIMO_PL_CLAVES();
```

Lógica de Control

Este trigger intercepta modificaciones en **CONTIENE** para validar dinámicamente si el artículo supera el límite.

Al utilizar una subconsulta directa, se computa de manera precisa el total de palabras asociadas al identificador modificado.

Si se excede la restricción, el trigger aborta la transacción lanzando una excepción controlada.

Bases de Datos II

Implementación de RI con Trigger

1. Función del Trigger — Con Variable Local (DECLARE)

```
CREATE OR REPLACE FUNCTION FN_MAXIMO_PL_CLAVES()
RETURNS TRIGGER AS $$
DECLARE
  cant INTEGER;
BEGIN
  SELECT count(*) INTO cant FROM CONTIENE
  WHERE id_articulo = new.id_articulo;
  IF (cant > 15) THEN
    RAISE EXCEPTION 'El artículo supera la cantidad de
    palabras: %', cant;
  END IF;
  RETURN NEW;
END $$ LANGUAGE 'plpgsql';
```

Uso de Variables de Estado

// Esta variante almacena temporalmente el resultado del conteo en la variable local **cant** *antes de realizar la evaluación lógica.*

2. Definición del Trigger

```
CREATE TRIGGER
TR_MAXIMO_PL_CLAVES
AFTER INSERT OR UPDATE OF
id_articulo
ON CONTIENE
FOR EACH ROW
EXECUTE PROCEDURE
FN_MAXIMO_PL_CLAVES();
```

Lógica de Control

Este trigger de tipo **row-level** intercepta modificaciones en la tabla **CONTIENE**.

Valida si el artículo excede el límite de **15 palabras clave** mediante un conteo almacenado en la variable **cant**.

Si se supera la restricción, aborta la transacción y muestra dinámicamente la cantidad detectada.

Bases de Datos II

Implementación de RI con Trigger

1. Función del Trigger (PL/pgSQL) — Variante BEFORE

```
CREATE OR REPLACE FUNCTION FN_MAXIMO_PL_CLAVES()
RETURNS TRIGGER AS $$
BEGIN
   IF ((SELECT count(*) FROM CONTIENE c
         WHERE id_articulo = new.id_articulo) ⩾ 15)
         THEN
     RAISE EXCEPTION 'El artículo supera la cantidad de
     palabras claves';
   END IF;
   RETURN NEW;
END $$ LANGUAGE 'plpgsql';
```

Nota sobre la Condición

// Al ejecutarse en modo BEFORE, debemos verificar si el conteo actual ya es mayor o igual al límite (>= 15) antes de aplicar la modificación.

2. Definición del Trigger

```
CREATE OR REPLACE TRIGGER
TR_MAXIMO_PL_CLAVES
BEFORE INSERT OR UPDATE OF
id_articulo
ON CONTIENE
FOR EACH ROW
EXECUTE PROCEDURE
FN_MAXIMO_PL_CLAVES();
```

Lógica de Eficiencia

La ejecución **BEFORE** resulta más conveniente para chequeos y restricciones en términos de rendimiento.

Evita el procesamiento innecesario de escrituras en disco al abortar la transacción antes de que se efectúe el cambio.

Precaución: Debe cuidarse la condición lógica ya que el estado 'NEW' aún no se ha consolidado en la tabla.

Bases de Datos II

Implementación de RI con Trigger

Restricción Declarativa RI Declarativa

Ejercicio: Sólo se pueden publicar artículos argentinos que contengan hasta 10 palabras claves

```
CREATE ASSERTION ASS_ART_ARG_MAX_PL
CHECK NOT EXISTS (
  SELECT 1
  FROM ARTICULO A JOIN CONTIENE C
  ON (A.id_articulo = C.id_articulo)
  WHERE nacional = 'Argentina'
  GROUP BY A.id_articulo
  HAVING COUNT(*) > 10
);
```

Modelo Relacional

ARTICULO
PK id_articulo: int
titulo: varchar(120)
nacional: char(10)

CONTIENE
FK id_articulo: int
FK codigo_palabra: smallint

PALABRA
PK codigo_palabra: smallint
descripcion: varchar(25)

Tabla / Evento	INSERT	UPDATE	DELETE
ARTICULO	NO	SI (nacional)	NO
CONTIENE	SI	SI (id_articulo)	NO

Bases de Datos II

Implementación de RI con Trigger

1. Trigger sobre ARTICULO

```
-- se activa ante cambios de nacionalidad

CREATE TRIGGER
TR_ART_ARG_MAXIMO_PL_CLAVES_ART

BEFORE UPDATE OF nacionalidad

ON ARTICULO

FOR EACH ROW

WHEN (NEW.nacional = 'Argentina')

EXECUTE PROCEDURE
FN_ART_ARG_MAXIMO_PL_CLAVES();
```

Nota: Solo se valida cuando un artículo existente es modificado y su nacionalidad pasa a ser 'Argentina'.

2. Trigger sobre CONTIENE

```
-- asocia palabras clave a los artículos

CREATE TRIGGER
TR_ART_ARG_MAXIMO_PL_CLAVES_CONT

BEFORE INSERT OR UPDATE OF id_articulo

ON CONTIENE

FOR EACH ROW

EXECUTE PROCEDURE

 FN_ART_ARG_MAXIMO_PL_CLAVES();
```

Nota: Se ejecuta siempre al insertar o asociar nuevas palabras clave a un artículo para garantizar el límite establecido.

Implementación de RI con Trigger



```
CREATE OR REPLACE FUNCTION FN_ART_ARG_MAXIMO_PL_CLAVES()
RETURNS TRIGGER AS $$
DECLARE
cant integer;
v_nacional articulo.nacional%type;
BEGIN
-- cuando el trigger se activa por un update en ARTÍCULO
IF (TG_TABLE_NAME = 'articulo') THEN
     SELECT count(*) INTO cant
     FROM CONTIENE
     WHERE id_articulo = new.id_articulo;

     IF (cant > 10) THEN -- se permite hasta 10 artículos argentinos
          RAISE EXCEPTION 'Supera la cantidad de palabras claves: %', cant;
     END IF;
ELSE ...
```

Bases de Datos II

Implementación de RI con Trigger

```
-- cuando el trigger se despierta por un insert or update en CONTIENE
SELECT nacional INTO v_nacional
FROM ARTICULO A
WHERE id_articulo = NEW.id_articulo;

IF (v_nacional = 'Argentina') THEN
   SELECT count(*) INTO cant
   FROM CONTIENE
   WHERE id_articulo = new.id_articulo;

   IF (cant ⩾ 10) THEN
      RAISE EXCEPTION 'Supera la cantidad de palabras claves: %', cant;
   END IF;
END IF;

RETURN NEW;
END $$ LANGUAGE 'plpgsql';
```

Bases de Datos II

Triggers vs. RI Declarativas

Uso de Triggers y RI

Definición Base:

Triggers permiten definir y forzar reglas de integridad (RI), pero ***NO son una RI*** por sí mismos.

Criterio de Aplicación:

Deben utilizarse **únicamente** cuando no sea posible expresar la restricción mediante una cláusula declarativa.

En tal caso, se deberá:

- Controlar la restricción mediante triggers aplicados en una o varias tablas.
- Analizar detalladamente los eventos críticos, granularidad, tiempo de activación y acciones correspondientes.

Consideraciones Críticas

Tener en cuenta que:

Verificación de Datos Existentes:

- Un trigger definido para forzar una RI **NO verifica** los datos ya almacenados en la base de datos. En cambio, una RI declarativa sí los valida al momento de su creación.

Alcance Operacional:

- La RI declarativa se fuerza automáticamente ante cualquier operación. Con triggers, es obligatorio declararlos e implementarlos explícitamente para todos los eventos críticos.

Flexibilidad y Control:

- Los triggers permiten implementar flujos de control complejos y acciones complementarias con un nivel de flexibilidad muy superior al de las RI declarativas estándar.

Bases de Datos II

Triggers - utilidad

Reglas de Integridad Complejas

Forzar reglas de integridad o de negocio complejas (cuando no es posible incluirlas declarativamente) o con acciones específicas de reparación.

Transición de Estados

Incorporar restricciones cuando se requiere conocer y comparar el valor anterior y el nuevo de los datos.

Generación de Logs

Creación automática de registros para soporte de auditoría de todas las acciones críticas sobre la base de datos.

Mantener Datos Derivados

Garantizar la consistencia mediante la propagación automática de actualizaciones y la generación dinámica de datos calculados ante cambios de estado.

Vistas Actualizadas

Mantener la sincronía de vistas complejas o materializadas de forma eficiente, especialmente cuando el DBMS no provee mecanismos automáticos nativos.

Bases de Datos II

RI de Transición con Trigger

Concepto & Restricciones

Definición:

Las RI de transición restringen los posibles cambios de valores entre **estados sucesivos** de los datos.

Mecanismo de Control:

Implica conocer y comparar el estado **"anterior" (old)** y el **"nuevo" (new)**.

Este tipo de RI no puede controlarse en forma declarativa, sino exclusivamente vía triggers.

Caso de Estudio:

Ej: La fecha de publicación de un artículo no puede adelantarse.

Implementación PL/pgSQL

```
CREATE OR REPLACE FUNCTION
fn_cntr_fecha_pub()
RETURNS trigger AS $$
BEGIN
  IF (new.fecha pub < old.fecha pub) THEN
    RAISE EXCEPTION 'La fecha no puede
    adelantarse';
  END IF;
  RETURN NEW;
END $$ LANGUAGE 'plpgsql';
```

Definición del Trigger

```
CREATE TRIGGER tr_fecha_pub
BEFORE UPDATE OF fecha_pub ON ARTICULO
FOR EACH ROW EXECUTE PROCEDURE
fn_cntr_fecha_pub();
```

Bases de Datos II

Trigger - Utilidad (Logs)

Modelo de Datos & Escenario

Objetivo: Registrar en una tabla de auditoría (`log_pal`) todas las modificaciones realizadas en la descripción de las palabras.

TABLA: PALABRA

```
codigo_palabra smallint PK
idioma char(2) PK
descripcion varchar(25)
```

TABLA: LOG_PAL (Auditoría)

```
usuario varchar(100)
fecha timestamp
val_old varchar(25)
val_new varchar(25)
```

Implementación en PostgreSQL

```
CREATE OR REPLACE FUNCTION fn_log_p()
RETURNS trigger AS $$
BEGIN
  INSERT INTO log_pal
  VALUES (current_user,
  current_timestamp, old.descripcion,
  new.descripcion);
  RETURN NEW; -- null si es statement
END; $$ LANGUAGE 'plpgsql';
```

Definición del Trigger

```
CREATE TRIGGER tr_log_pal
AFTER UPDATE OF descripcion ON
palabra
FOR EACH ROW
EXECUTE PROCEDURE fn_log_p();
```

Bases de Datos II

Trigger - Utilidad

Mantener automáticamente datos derivados: **cant_pal** en **ARTICULO**

Modelo de Datos

TABLA: ARTICULO

id_articulo PK
titulo, autor, fecha_pub, nacional
cant_pal integer (Derivado)

TABLA: CONTIENE (Relación)

id_articulo FK
codigo_palabra FK
idioma FK

TABLA: PALABRA

codigo_palabra PK, idioma PK
descripcion

Nota Importante:

Deberán especificarse otros triggers para **INSERT** y **DELETE**, o bien un único trigger usando **TG_OP**.

Implementación PL/pgSQL

```
CREATE FUNCTION fn_pal_update()
RETURNS trigger AS $$
BEGIN
  UPDATE articulo SET cant_pal =
  cant_pal + 1
  WHERE id_articulo = NEW.id_articulo;
  UPDATE articulo SET cant_pal =
  cant_pal - 1
  WHERE id_articulo = OLD.id_articulo;
  RETURN NEW;
END; $$ LANGUAGE 'plpgsql';
```

Definición del Trigger

```
CREATE TRIGGER tr_pal_update
AFTER UPDATE OF id_articulo ON contiene
FOR EACH ROW
EXECUTE FUNCTION fn_pal_update();
```

Bases de Datos II

Procedimientos Almacenados (Stored Procedures)

Concepto y Características

Ejecución Explícita:
Son bloques de código que se ejecutan a voluntad cuando son invocados por el usuario o la aplicación.

Gestión de Objetos:
Tienen la capacidad de crear, alterar o destruir objetos de la base de datos dentro de su propio bloque.

Sin Retorno de Datos:
No devuelven un valor de retorno a diferencia de las funciones tradicionales.

Parámetros:
Pueden recibir parámetros de entrada para parametrizar su comportamiento interno.

Sintaxis en PostgreSQL

```
CREATE OR REPLACE PROCEDURE
nombre_sp(args)
LANGUAGE plpgsql
AS $$
DECLARE
  -- declaración de variables
BEGIN
  -- cuerpo del procedimiento
END; $$;
```

Invocación

```
CALL nombre_sp(valores);
```

Origen https://www.postgresqltutorial.com/

Bases de Datos II

FN y SP: Funciones (Functions)

Concepto y Características

Ejecución Explícita:
Son bloques de código estructurado que se ejecutan a voluntad cuando son invocados explícitamente.

Gestión de Objetos:
Tienen la capacidad de crear, alterar o destruir objetos de la base de datos dentro de su propio ámbito.

Retorno Flexible:
A diferencia de los SP, las funciones **siempre** devuelven un valor, desde tipos simples hasta tablas completas.

Parámetros:
Soportan el paso de parámetros de entrada para parametrizar dinámicamente su comportamiento.

Sintaxis en PostgreSQL

```
CREATE [OR REPLACE] FUNCTION
fn_name(args)
RETURNS tipo_retorno
LANGUAGE plpgsql
AS $$
DECLARE
 -- declaración de variables
BEGIN
 -- lógica y consultas SQL
END; $$;
```

Ejemplo de Invocación

```
SELECT get_film_count(40, 90);
```

Origen https://www.postgresqltutorial.com/

Bases de Datos II

FN y SP: Funciones que devuelven Tablas

Estructura de Retorno

Permite definir funciones que retornan un conjunto de registros estructurados como una tabla virtual.

```
CREATE FUNCTION name(args)
RETURNS TABLE (cols)
LANGUAGE plpgsql
AS $$
DECLARE
-- variables
BEGIN
-- lógica de consulta
RETURN QUERY ...;
END; $$;
```

Ejemplo Práctico: get_film

```
CREATE OR REPLACE FUNCTION get_film(
p_pattern VARCHAR
)
RETURNS TABLE (
title VARCHAR,
release_year INT
)
LANGUAGE plpgsql AS $$
BEGIN
RETURN QUERY
SELECT film_title, year::int
FROM film WHERE film_title ILIKE p_pattern;
END; $$;
```

Invocación de la Función

```
SELECT * FROM get_film('Al%');
```

Origen https://www.postgresqltutorial.com/

FN y SP: Funciones que devuelven Tablas

Implementación con Query (get_film_query)

```
CREATE OR REPLACE FUNCTION
get_film_query(
  p_pattern VARCHAR,
  p_year INT )
RETURNS TABLE (
  film_title VARCHAR,
  film_release_year INT )
LANGUAGE plpgsql AS $$
BEGIN
  RETURN QUERY
    SELECT title, release_year FROM film
    WHERE title ILIKE p_pattern
      AND release_year = p_year
END; $$;
```

Uso

```
Select *
from get_film_query('%title%',2006)
```

film_title	release_year
TITLE 1	2006
TITLE 2	2006
TITLE 3	2006
TITLE 4	2006
TITLE 6	2006
TITLE 5	2006

Origen https://www.postgresqltutorial.com/

FN y SP: Funciones que devuelven Tablas

Implementación con Loop (get_film)

```
CREATE OR REPLACE FUNCTION get_film(
  p_pattern VARCHAR,
  p_year INT )
RETURNS TABLE (
  film_title VARCHAR,
  film_release_year INT )
LANGUAGE plpgsql AS $$
DECLARE
  var_r record;
BEGIN
  FOR var_r IN (
    SELECT title, release_year FROM film
    WHERE title ILIKE p_pattern
      AND release_year = p_year
  ) LOOP
    film_title := upper(var_r.title);
    film_release_year :=
var_r.release_year;
    RETURN NEXT;
  END LOOP;
END; $$;
```

film_title	release_year
TITLE 1	2006
TITLE 2	2006
TITLE 3	2006
TITLE 4	2006
TITLE 6	2006
TITLE 5	2006

Origen https://www.postgresqltutorial.com/

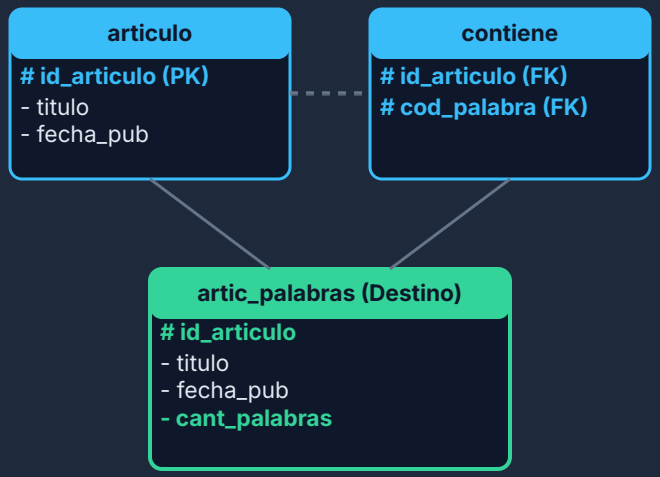
Bases de Datos II

FN y SP: Caso Práctico (Stored Procedures)

Enunciado del Problema

Se requiere completar la tabla **artic_palabras** con la cantidad de artículos publicados luego de una fecha dada y la cantidad de palabras asociadas.

Restricción: *No se permite el uso de bucles (FOR, LOOP). Debe resolverse puramente con sentencias SQL relacionales.*



Solución: Stored Procedure

```
CREATE OR REPLACE PROCEDURE
completar_artic_pal (
fecha DATE
)
LANGUAGE plpgsql AS $$
BEGIN
-- Limpiar datos previos
DELETE FROM artic_palabras;

-- Insertar consolidado de artículos
INSERT INTO artic_palabras
SELECT id_articulo, titulo, fecha_pub,
count(*)
FROM articulo
LEFT JOIN contiene USING (id_articulo)
WHERE fecha_pub > fecha
GROUP BY id_articulo, titulo, fecha_pub;
END; $$;
```

Invocación del Procedimiento

```
CALL completar_artic_pal('2023-04-01');
```

Bases de Datos II

Bibliografía de Consulta

01

“Fundamentals of Database Systems”

Elmasri, R., Navathe, S. — Addison Wesley, 2011 (Cap-5)

02

“Database System Concepts”

Silberschatz, A., Korth, H, Sudarshan, S. — McGraw Hill, 2002 (Cap-6)

03

Fundamentals of Relational Database Management Systems

Sumathi S., Esakkirajan S. — 2007 (Cap-5)

Documentación Oficial de PostgreSQL

GBD PostgreSQL — www.postgresql.org/docs/